AI on a Shoestring

Class Notes & Comprehensive Cheat Sheet

Champagne Intelligence on a Beer Budget: Local LLMs, Agents, RAG & Colibrì

Sudheer K. Mohammed · *BARC*

The Shoestring Creed: "Never rent in the cloud what your toaster can compute for free. If your AI monthly bill exceeds your grocery bill, you aren't doing engineering—you're paying a stupidity tax."

1. The Philosophy of Shoestring AI: Why Run Locally?

- **The Subscription Black Hole**: Commercial SaaS models lock basic capabilities behind $20/month per-user paywalls. Running locally restores zero-marginal-cost computing.
- **Token Anxiety & Financial Predictability**: Cloud APIs meter prompt tokens, completion tokens, and context caching. A runaway while-loop or bad agent recursive call can drain hundreds of dollars before dawn. Local inference costs $0.00 per token.
- **Absolute Privacy & Compliance**:
  - Proprietary source code, customer records, and personal documents never touch third-party servers.
  - Built-in immunity to model retraining on your private intellectual property.
- **Zero Latency Jitter & Offline Autonomy**: Functions in planes, trains, air-gapped server closets, or remote field sites without an internet connection.

2. Narrow vs. Broad AI: Right-Sizing Your Model

- **Broad / Frontier AI (70B – 1T+ parameters)**:
  - General-purpose behemoths trained on trillions of tokens.
  - Exceptional at high-abstract reasoning, multi-lingual poetry, complex thesis synthesis.
  - **The Penalty**: Demands multi-GPU clusters (8x H100s, 640GB+ VRAM) or massive cloud rental bills.
- **Narrow / Small Language Models (SLMs, 1B – 8B parameters)**:
  - Specialized powerhouses: **Phi-3.5** (Microsoft), **Qwen-2.5** (Alibaba), **SmolLM2** (Hugging Face), **Llama-3.2** (Meta).
  - With domain fine-tuning and task-specific instruction prompts, a 3B–7B model frequently matches or beats a 70B model on parsing, JSON schema extraction, and shell command generation.
  - **The Shoestring Rule**: Pick the smallest model that reliably satisfies your acceptance tests. Running a 70B model to lowercase a string is architectural malpractice.

3. AI Model Anatomy & Weight Formats

- **What is a Model File?** A collection of neural network weight tensors ($\mathbf{W}$) and bias vectors ($\mathbf{b}$), accompanied by architectural metadata (layer count, attention head dimensions, context window length) and a tokenizer vocabulary.

- **The Primary File Formats**:
  - **Safetensors (`.safetensors`)**:
    * Hugging Face open standard replacing insecure PyTorch `.bin`/`.pt` pickle files.
    * Zero arbitrary code execution risk, zero deserialization overhead, fast memory-mapping (`mmap`). Default for training and fine-tuning.
  - **GGUF (GPT-Generated Unified Format)**:
    * The undisputed universal format of consumer local inference, created by Georgi Gerganov (`llama.cpp`).
    * **Self-Contained**: Contains all model architecture metadata, tokenizer configuration, and quantized weights in a single binary file.
    * Supports split offloading between CPU system RAM and GPU VRAM seamlessly.
  - **ONNX & EXL2**:
    * **ONNX**: Standard for cross-platform edge inference (browsers, mobile, C#/.NET).
    * **EXL2**: Ultra-fast variable-bitrate quantization designed exclusively for high-throughput Nvidia GPUs via ExLlamaV2.

4. Model Quantization: Squeezing Giants into Laptops

- **Core Concept**: Neural networks are trained in 16-bit floating point (`FP16` = 2 bytes per weight). Quantization maps these continuous floats onto discrete low-bit integer bins (8-bit, 4-bit, or 2-bit).
- **Why it Works**: Neural network activations are intrinsically tolerant to stochastic noise. Reducing precision by 75% (from 16-bit to 4-bit) typically increases model perplexity (error) by less than 1.5%.
- **VRAM / RAM Calculation Formula**:

$$\text{Required Memory (GB)} \approx \text{Parameter Count (in Billions)} \times \frac{\text{Quant Bit-Depth}}{8} \times 1.25$$ (includes KV cache overhe

Quantization Type	Effective Bits	Size (7B Model)	Perplexity Delta	Shoestring Verdict
`FP16`	16.0 bits	14.0 GB	Baseline (0.00)	Only for GPU billionaires
`Q8_0`	8.5 bits	7.7 GB	Negligible (+0.01)	Archival quality
`Q5_K_M`	5.5 bits	5.1 GB	Almost imperceptible	Ideal if you have 16GB RAM
`Q4_K_M`	**4.5 bits**	**4.4 GB**	**Tiny (+0.08)**	**The Universal Sweet Spot**
`Q3_K_S`	3.2 bits	3.5 GB	Noticeable (+0.35)	Use only when severely constrained
`Q2_K`	2.6 bits	2.8 GB	Heavy degradation	Creative writing turns to soup

Colibrì Teaser: When Even 4-bit Won't Fit

The Dilemma: An 8B model at Q4 fits in 4.5GB RAM. But what if you want to run a **744B frontier MoE** model? At 4-bit, 744B still demands 370GB! How do you run that on a standard $800 laptop with 24GB of RAM?
→ *Answer: Sparse Mixture of Experts + NVMe Streaming. See Section 8.*

5. The Local Runtimes Trinity: llama.cpp, Ollama, LM Studio

- **1. llama.cpp (The Engine Room)**:
  - Written in pure C/C++ with SIMD optimizations (AVX2, AVX-512, Apple Metal, CUDA, ROCm, Vulkan).
  - Zero dependencies, compiles in seconds with `make` or `cmake`.

  - Supports fine-grained layer offloading: e.g., `-ngl 24` offloads 24 layers to GPU and keeps the rest in CPU RAM.
- **2. Ollama (The Docker of AI)**:
  - Wraps `llama.cpp` in a slick background daemon and CLI manager.
  - Automatically detects GPU capabilities, pulls quantized GGUF models, and manages memory.
  - Exposes a native REST API and an **OpenAI-compatible server** on port `11434`.
- **3. LM Studio (The Visual Powerhouse)**:
  - Desktop GUI (Electron/native) with built-in Hugging Face model search and downloader.
  - Real-time generation token/sec stats, GPU VRAM allocation slider, and local developer server.

Instant Terminal Mastery

```
# Install Ollama on Linux/macOS
curl -fsSL https://ollama.com/install.sh | sh

# Pull and interact with a lean coding model
ollama run qwen2.5-coder:3b "Write a Python script to sort a dictionary by
    value"
```

6. Navigating the Hugging Face Ecosystem

- **The Hub**: Central registry hosting open weights, datasets, benchmarks, and spaces.
- **Model Naming Conventions**: e.g., `Qwen/Qwen2.5-7B-Instruct-GGUF`
  - **Org / Creator**: `Qwen` (Alibaba), `meta-llama`, `microsoft`, `mistralai`.
  - **Parameter Size**: `3B`, `7B`, `14B`, `70B`.
  - **Tuning Type**: `-Base` (raw pretraining), `-Instruct` / `-Chat` (RLHF/DPO aligned).
  - **Format**: `-GGUF` (ready for Ollama/llama.cpp).
- **Key Community Quantizers**:
  - **bartowski**: The benchmark gold-standard for rapid, high-precision GGUF conversions.
  - **TheBloke**: Legendary community pioneer who quantized thousands of open models.
- **Open LLM Leaderboard**: Filter by evaluation benchmarks (MMLU-Pro, GSM8k, MATH, IFEval) to identify models that punch above their parameter class.

7. Architectures: Dense Models vs. Mixture of Experts (MoE)

- **Dense Models**: Every token passes through 100% of the feed-forward network weights. (e.g., Llama-3-8B executes all 8 billion parameters on every word).
- **Sparse Mixture of Experts (MoE)**:
  - The dense feed-forward layer is divided into E distinct expert sub-networks (e.g., 8, 16, or 64 experts).
  - A tiny, learned **Router Network** evaluates the token embedding and routes it to only the Top-K experts (typically $K = 2$).
  - **Example**: Mixtral 8x7B has 47B total parameters, but only activates ~13B parameters per token.
  - **The Architectural Epiphany**: Since 85%–95% of weights sit idle for any given token, **why must they all reside in expensive VRAM?**

8. Spotlight Study: The Colibrì Project (Extreme Shoestring)

The Colibrì Architecture: Pure C MoE Engine

The Mission: Shatter the assumption that running a 744-Billion parameter frontier model requires an enterprise datacenter rack.

- **Language**: Handcrafted in pure C for raw memory and disk I/O control.
- **Hardware Target**: A standard $800 consumer laptop with only **24GB–25GB of RAM** and no dedicated GPU server rack.
- **The 3-Tier Memory Hierarchy**:
  1. **Tier 1: System RAM (~10–15 GB)**: Pins the shared dense attention layers, token embeddings, normalization weights, and router permanently into fast memory.
  2. **Tier 2: High-Speed NVMe SSD (~370 GB)**: Houses the tens of thousands of routed expert parameters compressed in 4-bit quantization on disk.
  3. **Tier 3: Dynamic On-Demand Streaming + LRU Cache**: As the model decodes token-by-token, the router identifies the 2 active experts. Colibrì streams those exact weights from the NVMe SSD into an LRU cache in RAM in real time!
- **Performance vs. Freedom**:
  - Generation speed ranges from **0.5 to 1.8 tokens/sec**.
  - Is it instant? No. But it performs complex, frontier-level document analysis, code audit, and research locally, with zero cloud dependency and zero telemetry.

9. Connecting Local Models to Python

- **Direct Ollama Python Library**:

```python
import ollama

res = ollama.chat(
    model="llama3.2:3b",
    messages=[{"role": "user", "content": "Explain RAM paging in one sentence."
        }]
)
print(res["message"]["content"])
```

- **Universal OpenAI API Drop-in**: Any code written for the official OpenAI SDK runs unchanged against Ollama:

```python
from openai import OpenAI

client = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
completion = client.chat.completions.create(
    model="qwen2.5:3b",
    messages=[{"role": "user", "content": "Return the first 5 prime numbers as a
        JSON array."}]
)
print(completion.choices[0].message.content)
```

10. Local Autonomous Agents & Tool Use (ReAct)

- **The ReAct (Reason + Act) Loop**:
  1. **Prompt Injection**: User provides a high-level goal and available Python tools.

2. **Reasoning**: Model outputs structured tool intent: `{"tool": "query_db", "args": {"id": 42}}`.
3. **Execution**: Python executes the function locally in the sandbox.
4. **Feedback**: Tool observation is appended to message history.
5. **Resolution**: Model evaluates whether the goal is completed or another action is required.

- **Deterministic JSON with Pydantic**: Small models can drift in formatting. Enforce strict JSON output using Ollama's schema validator:

```
from pydantic import BaseModel

class ToolCall(BaseModel):
    command: str
    target_file: str

res = ollama.chat(
    model="qwen2.5:3b",
    messages=[...],
    format=ToolCall.model_json_schema()
)
```

- **Zero-Cost Loops**: Multi-step agent problem solving that would cost $5 in cloud tokens costs nothing locally.

11. RAG on a Shoestring: Local Private Retrieval

- **Why RAG?** LLMs suffer from context limits and hallucinations. RAG grounds responses in your private local documents.
- **Local Vector Databases**:
  - **ChromaDB**: Serverless, lightweight, persists to a local directory or runs in-memory.
  - **FAISS (Facebook AI Similarity Search)**: Pure C++/Python vector search for millions of embeddings.
- **Shoestring Embedding Models**:
  - `bge-small-en-v1.5` (33M params, ~130MB): Outperforms OpenAI Ada-002 on retrieval benchmarks.
  - `nomic-embed-text` (available directly inside Ollama with `ollama pull nomic-embed-text`).

12. Complete End-to-End Demo: Prompt to Result

```
import json, ollama
from pydantic import BaseModel, Field

# 1. Define Strict Pydantic Schema
class ServerIncident(BaseModel):
    service: str = Field(description="Name of crashed service")
    error_code: int = Field(description="HTTP or kernel error code")
    recommended_action: str = Field(description="Actionable fix")

# 2. Raw Unstructured Log Input
raw_log = """
[ERROR] 2026-10-16 21:05:12.341 [auth_service] worker-3 crashed!
Code: 504 Gateway Timeout while contacting Redis cluster at 10.0.4.12.
Recommend immediate restart of sentinel node.
"""

# 3. Local Model Structured Extraction
```

```
response = ollama.chat(
    model="qwen2.5:3b",
    messages=[
        {"role": "system", "content": "Extract server incident reports into strict
            JSON."},
        {"role": "user", "content": raw_log}
    ],
    format=ServerIncident.model_json_schema()
)

# 4. Validate and Execute Tool in Python
incident = ServerIncident.model_validate_json(response['message']['content'])
print(f"AUTOMATED ALERT: {incident.service} (Code {incident.error_code})")
print(f"ACTION REQUIRED: {incident.recommended_action}")
```

13. Local Coding Copilots & Evaluation Harnesses

- **Continue.dev (VS Code & JetBrains)**:
  - Replaces GitHub Copilot completely.
  - Configure Tab-Autocomplete with an ultra-fast 1.5B model (e.g. `qwen2.5-coder:1.5b`) for $<$ 30ms latency.
  - Configure Chat & Inline Refactor with a 7B model (e.g. `qwen2.5-coder:7b`).
- **Aider (Terminal Pair Programmer)**:
  - Interacts with git repositories directly from the shell: `aider -model ollama/qwen2.5-coder:7b`.
  - Automatically reads git diffs, edits multi-line files, and crafts concise git commit messages.
- **Local Test Evaluation Harness**:
  - Create an automated `pytest` suite with 20–50 fixed prompts.
  - Benchmark new model releases locally on schema adherence, code syntax validity, and tokens/sec before swapping models.

14. Building Custom Standalone Applications

- **Architecture**: FastAPI (API backend) + Streamlit or Gradio (Web UI) + Ollama/llama.cpp (Inference backend).
- **Deployment Strategy**:
  - Containerize with a single multi-stage `Dockerfile`.
  - Deploy on edge devices (Raspberry Pi 5 with 8GB, Intel NUC, or old office desktop).
  - Perfect for air-gapped hospitals, legal offices, and industrial controllers.

15. AI for Supercharged Personal Productivity

- **Terminal Shell Translator**: Pipe messy tasks to local models instead of scouring documentation.
- **Private Audio Meeting Transcriber**: Run `whisper.cpp` (or `insanely-fast-whisper`) locally to convert 1-hour voice recordings to text in 2 minutes, then summarize with `llama3.2:3b`.
- **Local Git Commit Generator**:

```
git diff --cached | ollama run qwen2.5-coder:3b "Write a 1-line git commit
    message in conventional commit format:"
```

16. Hardware Sizing Cheat Sheet & Shoestring Rules

Hardware Setup	Recommended Model	Quantization	Typical Speed
Standard Laptop (8GB RAM, CPU)	Llama-3.2-3B or SmolLM2	`Q4_K_M`	15–22 t/s
Mid-Tier PC (16GB RAM, integrated graphics)	Qwen2.5-7B or Llama-3.1-8B	`Q4_K_M`	8–14 t/s
Budget GPU (RTX 3060 12GB VRAM)	Qwen2.5-Coder-7B (Full VRAM)	`Q8_0` or `Q5_K_M`	55–75 t/s
Apple Silicon (M1/M2/M3 Pro/Max 36GB+)	Qwen2.5-32B or Command-R	`Q4_K_M`	18–30 t/s
Colibrì Mode (24GB RAM + Gen4 NVMe)	**GLM-4/5.2 MoE (744B!)**	`Q4` SSD Stream	0.5–1.8 t/s

Three Deadly Sins of Local AI

1. **The 128k Context Trap**: Expanding context window to 128k tokens allocates 8GB+ purely for the KV-cache. Keep default context to 4k or 8k unless needed.
2. **Unquantized Weights (`FP16`)**: Attempting to run raw FP16 weights on consumer hardware triggers memory thrashing, swap thrashing, and frozen laptops.
3. **Benchmark Paralyzation**: Spending 40 hours benchmarking 15 different 7B models instead of building your actual agent workflow. Pick `Qwen2.5` or `Llama3.1` and start shipping!

— End of Class Notes · Compiled Offline with 100% Free Open-Source Software —